

INSTITUTO POLITÉCNICO FORMOSA

TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

TRABAJO PRÁCTICO INTEGRADOR

Diseño e Implementación de Soluciones Móviles Híbridas con React Native y Expo

Asignatura:	Taller de Lenguajes de Programación III
Estudiante:	Fariña Eric Andres
Temática:	Opción B - Catálogo Personal de Medios (Cine, Libros y Series)
Tecnología:	Expo SDK 54 (Nueva Arquitectura)
Fecha de Entrega:	Junio, 2026
Localización:	Formosa, Argentina

Sección I: Introducción al Desarrollo Móvil Híbrido

Conceptualización de React Native y Mecanismos de Renderizado

React Native es un framework de código abierto creado por Meta que permite el desarrollo de aplicaciones móviles multiplataforma utilizando JavaScript y la biblioteca React. A diferencia de otros esquemas de desarrollo que buscan la unificación de código, el paradigma de React Native no consiste en "escribir una vez y ejecutar en cualquier lugar", sino en "aprender una vez y escribir en cualquier lugar".

Su ventaja radica en el proceso de renderizado. En las versiones más modernas del framework (como las implementadas a partir de Expo SDK 54 y React Native 0.76), el motor opera bajo la **Nueva Arquitectura (New Architecture)**. Esto significa que el antiguo "Bridge" asíncrono que comunicaba JavaScript con el sistema nativo ha sido reemplazado por **JSI (JavaScript Interface)** y el motor de renderizado **Fabric**. JSI permite que el hilo de JavaScript invoque métodos nativos de iOS y Android de forma síncrona y directa. Por ende, un elemento estructural no se dibuja como un píxel en un navegador embebido, sino que se traduce en un widget real del sistema operativo, como un `UIView` en iOS o un `android.view.View` en Android, logrando un rendimiento sin precedentes.

Diferenciación Explícita frente a Soluciones Basadas en WebView

Es fundamental distinguir a React Native de los frameworks híbridos de primera generación basados en WebViews (tales como Apache Cordova, PhoneGap o las versiones iniciales de Ionic).

Característica	Soluciones Basadas en WebView (Cordova / Ionic Antigua)	Arquitectura React Native (JSI / Fabric)
Entorno de Ejecución	Navegador web interno e invisible (WebView) integrado en la app.	Hilo JavaScript independiente interactuando con componentes nativos.
Renderizado de UI	DOM de HTML5, CSS manipulado por un motor de renderizado web.	Componentes de la interfaz de usuario nativos del sistema operativo.
Rendimiento Crítico	Limitado por las capacidades del navegador; propenso a tirones en animaciones.	Nativo real, renderizado síncrono optimizado a 60/120 FPS.

Característica	Soluciones Basadas en WebView (Cordova / Ionic Antigua)	Arquitectura React Native (JSI / Fabric)
Acceso a APIs de Hardware	Requiere plugins intermediarios pesados para traducir llamadas Web.	Llamadas directas síncronas mediante TurboModules (C++).

El Ecosistema Expo y sus Ventajas Operativas

Expo es un ecosistema y una plataforma compuesta por herramientas, librerías y servicios diseñados sobre React Native para simplificar radicalmente el ciclo de vida de desarrollo, construcción y despliegue de aplicaciones móviles.

Dentro de este ecosistema, **Expo Go** cumple el rol de cliente de pruebas de alto rendimiento. Se trata de una aplicación contenedora precompilada disponible en las tiendas oficiales (Google Play y App Store) que permite ejecutar el código fuente en desarrollo de manera instantánea mediante el escaneo de un código QR. Expo Go elimina la necesidad de compilar binarios nativos locales en cada modificación, abstrayendo al desarrollador de entornos complejos como Android Studio o Xcode durante las etapas iniciales e intermedias.

La comparación operativa entre la interfaz de línea de comandos de Expo (**Expo CLI**) y el flujo tradicional de **React Native CLI** evidencia ventajas críticas para el desarrollador moderno:

- **Abstracción del código nativo:** React Native CLI requiere la gestión directa de carpetas `/ios` y `/android`. Expo CLI oculta esta complejidad, autogenerando las capas nativas de forma limpia mediante *Prebuild* y configuraciones centralizadas en el archivo `app.json`.
- **Despliegue Multiplataforma Agnóstico:** Con React Native CLI es estrictamente obligatorio poseer macOS para compilar versiones de iOS. Expo CLI soluciona esto a través de EAS (Expo Application Services), permitiendo compilar en la nube desde entornos Linux o Windows.
- **Actualizaciones Over-The-Air (OTA):** Expo permite inyectar actualizaciones de JavaScript directamente en las aplicaciones instaladas de los usuarios sin pasar por las revisiones de las tiendas de aplicaciones.

Sección II: Anatomía de Componentes Core y Optimización

Mapeo de Componentes Core y su Equivalencia Web

Para estructurar la interfaz móvil, React Native provee abstracciones semánticas que reemplazan las etiquetas estructurantes del desarrollo web convencional:

- **<View>:** Equivale a la etiqueta estructural de bloque `<div>` en HTML. Actúa como un contenedor multipropósito para aplicar layouts estructurales y definir estilos.

- **<Text>**: Se equipara a etiquetas de texto como `<span>` o `<p>`. En React Native, cualquier cadena de texto cruda obligatoriamente debe estar contenida dentro de un componente `<Text>`.
- **<TextInput>**: Corresponde al elemento interactivo `<input type="text">` o `<textarea>` de HTML, permitiendo la captura de datos por teclado.

Análisis Crítico de Rendimiento: FlatList vs. ScrollView

La renderización de colecciones extensas es uno de los cuellos de botella más críticos. El uso imperativo de FlatList sobre un bucle iterativo `.map()` dentro de un ScrollView responde a una justificación de arquitectura.

Cuando se utiliza un ScrollView, el dispositivo procesa, calcula el layout e intenta instanciar la totalidad de los elementos del arreglo en memoria de forma simultánea. Si el arreglo contiene 1,000 elementos, consumirá recursos de manera lineal $O(N)$, degradando el rendimiento.

FlatList resuelve esta problemática implementando dos técnicas de optimización avanzadas:

1. **Renderizado Perezoso (Lazy Rendering)**: FlatList solo monta y renderiza los elementos del arreglo que son visibles en la interfaz en ese preciso instante, sumando un pequeño margen controlado (buffer).
2. **Reciclaje de Celdas en Memoria (Cell Recycling)**: A medida que un elemento sale del área visible, FlatList reutiliza esa misma estructura de contenedores visuales vacíos en memoria para poblarla con los datos del nuevo elemento que ingresa por la pantalla. Mantiene un consumo de memoria constante $O(1)$.

Sección III: Gestión de Estados e Interactividad

Reactividad con useState y el Mecanismo de Renderizado

El estado es una estructura de datos que almacena los valores dinámicos de un componente. Si un desarrollador mutara directamente una variable de JavaScript (ej. `arreglo.push()`), la pantalla no se actualizaría porque React no implementa observadores activos sobre variables locales arbitrarias.

El gancho `useState` proporciona el valor actual del estado y una función despachadora (función `set`). La función `set` reemplaza el estado con una nueva referencia (inmutabilidad) y envía una señal explícita al motor de React para que ejecute el algoritmo de reconciliación y actualice la pantalla de forma selectiva.

Control de Efectos y Sincronización mediante useEffect

El gancho `useEffect` permite la ejecución de efectos secundarios, regulado por su **matriz de dependencias**:

- **Sin matriz:** El efecto se ejecuta en cada renderizado (riesgo de bucles infinitos).
- **Matriz vacía []:** Se ejecuta única y exclusivamente una sola vez tras el montaje inicial.
- **Matriz con variables [estado]:** Se dispara selectivamente cuando dichas variables cambian.

En esta aplicación, `useEffect` fue implementado con una matriz vacía para inyectar datos de prueba (mock data) al montarse la app, asegurando que el `FlatList` no inicie vacío.

Sección IV: Diseño y Flexbox en Dispositivos Móviles

Comportamiento del Layout y Flexbox

El motor de maquetación en React Native utiliza **Yoga**, una implementación de Flexbox en C++. Algunas restricciones importantes son:

- Todos los contenedores poseen implícitamente `display: 'flex'`.
- Las dimensiones no usan unidades web (como 'px' o 'rem'), sino píxeles independientes de la densidad (dp), que Yoga escala automáticamente según la pantalla del dispositivo.

Análisis de flexDirection: Diferencia Web vs. Móvil

En el desarrollo Web convencional, el valor por defecto de `flexDirection` es `row` (fila), apilando elementos horizontalmente debido al formato de los monitores de escritorio.

En **React Native**, el valor por defecto absoluto de `flexDirection` es **column** (columna), apilando elementos de forma vertical. Esta decisión arquitectónica se fundamenta en la usabilidad móvil: los smartphones se manipulan en orientación vertical (portrait), provocando que la navegación natural sea descendente.